FPGA Accelerator for CNN-Based Image Super-Resolution on the ZedBoard (Zynq-7000 XC7Z020)

Major Project – Progress Report

Sreevenkat

Roll No.: 230104024

Guide: Dr. Nagesh Ch

Department of Electronics and Communication Engineering

Abstract

A ×2 image super-resolution CNN runs as a hand-written Verilog accelerator on a Digilent ZedBoard (Xilinx Zynq-7000 XC7Z020) and upscales a 960 × 540 image to 1920 × 1080 in **1.31 s** (board-measured, 100 MHz). Every output completed on the board so far is **byte-exact** with a bit-accurate INT8 software reference model. The 2,620-parameter network is trained in PyTorch, quantized to INT8 and executed tile by tile in the FPGA fabric; the ARM processor feeds the tiles through AXI DMA. The full system uses 68 DSP slices (31 %), 72 block RAMs (51 %) and 5,146 LUTs (10 %) and meets timing at 100 MHz. INT8 quality is 35.02 / 31.40 / 30.53 dB PSNR-Y on Set5 / Set14 / BSD100, i.e. 0.97–1.35 dB above bicubic interpolation and within 0.08–0.18 dB of the floating-point network.

1 Introduction and Scope

Super-resolution (SR) rebuilds a high-resolution (HR) image from a low-resolution (LR) one. CNN-based SR clearly beats interpolation [1, 2, 3], but even a small network needs about 1.3 billion multiply-accumulates (MACs) per 960 × 540 frame. A Zynq SoC lets the FPGA fabric do this arithmetic with fixed, predictable latency while the ARM cores handle control and memory.

The objective is a ×2 SR accelerator in hand-written RTL on the low-cost XC7Z020 whose output is bit-identical to a software reference. The original 4K target was dropped: with 220 DSP slices the device cannot reach a useful 4K frame rate with this approach. The main goal is 960 × 540 → 1920 × 1080 (×2, so the output fits the ZedBoard HDMI port); 64 × 64 → 128 × 128 is used for development, and ×4 (480 × 270 → 1080p) is a stretch goal. The work follows nine milestones: M0 Zynq bring-up (ARM, AXI, DMA), M1 software baseline, M2 FP32 training, M3 INT8 quantization, M4 convolution engine in RTL, M5 full tile core, M6 board system, M7 measurement and report, M8 optional extensions. Section 10 gives the status of each.

2 Related Work

SRCNN [1] showed that a three-layer CNN outperforms interpolation for SR. FSRCNN [2] removed the bicubic pre-upscaling and works directly on the LR image. ESPCN [3] introduced the sub-pixel convolution (pixel shuffle) used here for upsampling. Integer-only inference with a fixed-point multiplier and shift [4] is the basis of the INT8 scheme, and depthwise/pointwise convolutions [5] keep the network small. Two recent papers implement SR on Zynq-class FPGAs:

- **Ahmed et al.** [6] deploy an FSRCNN-style network on a low-cost Zynq-7000 board using Vivado HLS, loop unrolling and INT8 quantization (81 DSPs, 98 % of BRAM, 84 % of LUTs).
- **Joseph and Jayakumar** [7] train a quantized FSRCNN (W8A8) with Brevitas and generate a streaming dataflow design with FINN on a ZCU104 (Zynq UltraScale+). The network processes only the luma (Y) channel and reaches 24.12 fps for 1080p→4K using 1,114 DSPs.

This project differs in three ways: the accelerator is written by hand in Verilog (not HLS or FINN); it processes full RGB on the smallest common Zynq device; and every hardware result is checked bit for bit against a software integer model, from single layers up to complete frames on the board. Park et al. [8] describe an FPGA ray-tracing accelerator; it relates to the planned extension in which the input comes from a hardware ray tracer (M8), not to the current SR core.

3 System Overview

The work is split between a PC flow that produces the network and its reference results, and the ZedBoard that runs it (Fig. 1). On the board, the ARM pads the image and cuts it into tiles; the AXI DMA streams each tile from DDR into the accelerator and the upscaled tile back. Images move between PC and DDR over JTAG.

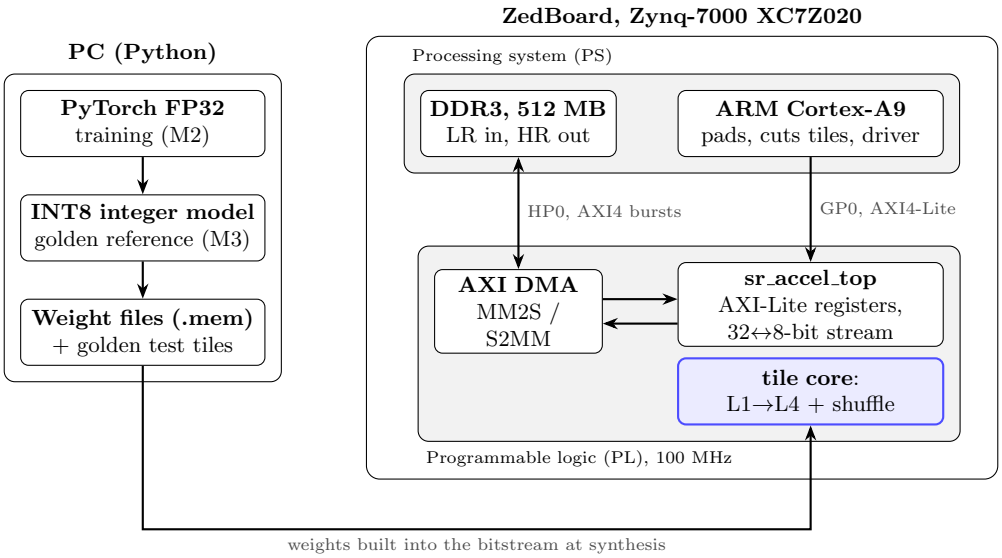


Figure 1: System architecture: PC flow (left) and ZedBoard processing system / programmable logic (right).

4 Network Design and Training

The network has four convolution layers, 2,620 parameters and 2,560 MACs per LR pixel (about 1.3 billion MACs per 960×540 frame). It predicts 12 channels per LR pixel (a 2×2 block $\times$ 3 colours), which pixel shuffle [3] rearranges into the $\times 2$ RGB image without any arithmetic.

Layer	Operation	Channels	Activation	Parameters	MACs / LR pixel
L1	Conv 3×3	$3 \to 16$	ReLU	448	432
L2	Depthwise conv 3×3	16	ReLU	160	144
L3	Pointwise conv 1×1	$16 \to 16$	ReLU	272	256
L4	Conv 3×3	$16 \to 12$	none	1,740	1,728
–	Pixel shuffle $\times 2$	$12 \to 3$	–	0	0
Total				**2,620**	**2,560**

Table 1: Network architecture.

All convolutions are *valid* (no padding). The whole input image is replicate-padded by 3 pixels once, because the three 3×3 layers each consume one pixel per side; this makes tiled hardware processing identical to whole-image processing. Only hardware-friendly operations are used: ReLU, no batch normalization, no residual additions. Training used 180 DIV2K [12] images (12 more for validation), random 38×38 LR windows with their 64×64 HR targets, flips and 90° rotations, L1 loss and Adam (learning rate 2×10^{-3} with cosine decay) for 100 epochs of 500 steps. A 24-channel model, an SSIM loss term and edge-focused sampling gave no meaningful gain. The FP32 network beats bicubic on all 119 test images (Table 2).

5 INT8 Quantization and the Integer Reference Model

Post-training INT8 quantization costs only 0.08–0.18 dB PSNR-Y, so no quantization-aware retraining was needed. Following [4]: unsigned 8-bit activations, symmetric signed 8-bit weights with one scale per layer, 32-bit biases and a 32-bit accumulator. Each output is requantized with a 16-bit integer multiplier M and a shift (21–24 bits), rounded to nearest and clamped:

$$y = \mathrm{clamp}\left(\left\lfloor \frac{\mathrm{acc} \cdot M + 2^{\,\mathrm{SHIFT}-1}}{2^{\,\mathrm{SHIFT}}} \right\rfloor ,\ 0,\ 255 \right), \qquad \mathrm{acc} = b + \sum a_i w_i. \tag{1}$$

Activation scales come from the 99.99th percentile of 100 training-image crops (the 99th percentile lost 5.3 dB; per-channel weight scales were slightly worse than per-layer). A pure-integer NumPy model reproduces exactly this arithmetic and is the **golden reference** for all hardware. It matches an independent float64 reference within 1 LSB, and tiled processing matches whole-image processing exactly.

Test set	Bicubic	FP32 CNN	INT8 model	Gain over bicubic (INT8)	FP32→INT8 drop
Set5 [9]	33.67	35.21	35.02	+1.35	0.18
Set14 [10]	30.32	31.51	31.40	+1.08	0.11
BSD100 [11]	29.56	30.60	30.53	+0.97	0.08

Table 2: Mean PSNR on the Y channel (dB), ×2.

SSIM-Y drops by about 0.003 from FP32 to INT8 (Set5: 0.9449 → 0.9420; bicubic 0.9303). The INT8 model beats bicubic on 119 of 119 test images. Because the board output is byte-exact with the INT8 model, the FPGA output has the same quality; the board run over all 119 test images is in progress (Section 10).

6 Hardware Architecture

The accelerator processes the image in 64×64 tiles, each loaded with a 3-pixel halo ($70 \times 70 \times 3$ bytes) and returned as a $128 \times 128 \times 3$ RGB tile. One tile takes 926,201 clock cycles (simulated), of which 7.8 % is byte-wide input and output.

Convolution engine. One parameterized engine per layer (C_{in}, C_{out}, kernel size, depthwise mode). It reads one activation per cycle and broadcasts it to up to 16 parallel MAC lanes, one DSP48E1 each, which accumulate all output channels at once. A single shared requantizer per engine works on pixel n while the MAC lanes already accumulate pixel n+1, which saves DSPs.

Layer	Channels	Cycles per LR pixel	DSP48E1	Tile size in → out
L1 conv 3×3	$3 \to 16$	27	18	$70 \to 68$
L2 depthwise 3×3	16	16	18	$68 \to 66$
L3 pointwise 1×1	$16 \to 16$	16	18	$66 \to 66$
L4 conv 3×3	$16 \to 12$	144	14	$66 \to 64$

Table 3: Per-layer engines.

Tile core. A byte-stream loader fills the first activation RAM. A sequencer runs L1–L4 back to back through five activation RAMs (one per layer boundary, pixel-major: one 128-bit word holds all 16 channels of a pixel). Pixel shuffle is pure address arithmetic in the output packer, which streams the 128×128 RGB tile in raster order. Because the convolutions are valid, the tile shrinks from 70 to 64 by itself and no cropping logic is needed.

System integration. `sr_accel_top` wraps the core with an AXI4-Lite register block (control, status, cycle counter, error code, version, debug counters) and converts the 32-bit AXI4-Stream of the Xilinx AXI DMA [14] to and from the core's byte stream. The DMA reads and writes DDR through the PS high-performance port HP0; the registers sit on the general-purpose port GP0 [13]; everything runs at 100 MHz. The weights are stored in on-chip ROMs initialized from the exported files at synthesis time.

ARM driver. Bare-metal C code pads the image, cuts the 135 tiles of a 960×540 frame (padded to 960×576), runs each through DMA and core with cache flush/invalidate and timeouts, and places the output tiles into the 1920×1080 frame. On any error or timeout it resets the DMA and the core and reports the cause.

7 Verification Methodology

One reference chain is used throughout: PyTorch FP32 → integer NumPy model → RTL → board. Hardware is always compared with the integer model, byte for byte.

1. **Unit level:** the requantizer is checked against 5,000–13,000 Python vectors per shift value (rounding ties, saturation); the MAC lane with random and extreme sequences.
2. **Layer level:** each engine is compared with the golden dump of its layer on 10 golden tiles (zeros, all-255, noise, single pixel, ramp, real tiles, border tile), plus randomized layers with per-channel multipliers, extreme weights and non-square tiles.
3. **Tile level:** inside the chained core, every intermediate RAM (L1–L4) and the output stream are compared after each tile, also with random stalls on both streams.
4. **Seam test:** a real image region is processed whole by the integer model and tile by tile by the RTL; the assembled result is identical (20 tiles at a small tile size, 4 at full size).
5. **Post-synthesis:** the synthesized netlists are simulated with the same testbenches.
6. **Mutation tests:** deliberate bugs (rounding, shift, byte order, kernel flip, handshake, reset, ...) must be caught: 22 in the integer model, 32 in the engine, 21 in the core, 20 in the AXI wrapper.
7. **Board level:** golden tile, 128×128 image and full 960×540 image compared byte for byte, with repeat runs, a power cycle, recovery from deliberately broken tiles and soak tests.

The automated regression has a quick set of 36 runs (≈2 min) and a full set of 88 runs (≈20 min).

8 Results

The board produces correct 1080p frames at 1.31 s each, within 5 % of the 1.25 s predicted from simulation, using under a third of the chip's DSPs.

Design	LUT	FF	BRAM tiles	DSP48E1	WNS
SR core + AXI wrapper (out of context)	1,988	3,479	70	68	+0.677 ns
Full system: PS + AXI DMA + accelerator	5,146 (9.7 %)	7,605 (7.1 %)	72 (51.4 %)	68 (30.9 %)	+0.239 ns

Table 4: Resources and timing after place and route (Vivado 2024.1, 100 MHz).

Vivado estimates 1.92 W on-chip power (low-confidence estimate, not measured), i.e. about 2.5 J per frame. Clocks above 100 MHz have not been tried yet.

Quantity (board-measured, 960 × 540 → 1920 × 1080, 15 frames)	Value
Frame time	1,312 ms (0.76 fps), spread <2 ms
DMA + compute	1,280 ms
ARM tile copy	26.5 ms
Image padding	5.9 ms
Cycles per tile on the board (simulation: 926,202)	926,217 – 926,220
Prediction: 135 tiles × 926,202 cycles at 100 MHz	1,250 ms

Table 5: Speed.

Correctness on the board. A golden tile is byte-exact (20 of 20 repeats identical). A 128 × 128 image and the 960 × 540 image are byte-exact; the full frame repeated 3 times plus 15 timing frames gave identical output and the same checksum after a power cycle. 8,000 tiles in soak tests gave 0 errors. Deliberately broken tiles are rejected with an error code, and the next tile is correct again.

	This work	Ahmed et al. [6]	Joseph and Jayakumar [7]
Device	XC7Z020 (ZedBoard)	Zynq-7000 (Cora board)	ZCU104 (Zynq UltraScale+)
Design method	Hand-written Verilog	Vivado HLS	FINN dataflow
Network	4-layer CNN, 2,620 param., RGB	FSRCNN-style, ×4	QFSRCNN, 6,065 param., Y only
Precision	INT8	INT8	W8A8
DSPs / LUTs	68 / 5,146	81 / 45,044	1,114 / 55K
Clock	100 MHz	not stated	214.53 MHz
Throughput	0.76 fps, 960 × 540 →1080p	"4.35 ns processing time" (as reported)	24.12 fps, 1080p→4K
Set5 PSNR, ×2	35.02 dB (Y)	not reported	35.40 dB

Table 6: Comparison with reference designs (different boards, networks and evaluation set-ups: design points, not a ranking).

This design uses far fewer resources than both references and is currently much slower than [7]: one engine per layer computes pixels one after another. Section 10 lists the planned speed-ups.

9 Problems Found and Solved

Most problems passed simulation and appeared only in synthesis or on the board, which is why each stage has its own checks. The last cause was found by controlled experiments: a simulated sweep of every timing offset between the start command and the first data word (918 tiles, all correct) ruled out an RTL race; board runs with and without isolation reproduced and removed the failure twice. Two single failures seen earlier remain unexplained and are monitored with hardware debug counters.

10 Status and Remaining Work

Milestones M0–M6 are complete: the accelerator works on the board and is verified end to end. M7 is in progress.

Problem	Root cause	Fix
Engine almost removed by synthesis (2 DSPs instead of 18)	ROM layout built in an `initial` loop, ignored by synthesis	Exporter writes the ROM layout; plain `$readmemh`
$70 \times 70 \times 128$-bit buffer used 32 BRAM36	RAM depth rounded up to a power of two	Split RAM (4096 + rest): 18.5 tiles
Slack only +0.135 ns at 100 MHz	Long mux path into the requantizer	One pipeline register: +2.9 to +3.9 ns
Accelerator rejected in block design	Compile-time check referred to a missing module	Check moved to testbench and synthesis script
Risk of empty weight ROMs	Relative path to the weight files	Absolute path; 12 "read successfully" messages, 68 DSPs
ARM timer always read 0	Global timer not started	Driver enables and reads it directly
New program corrupted after reload	Stale cache lines written back over it	L2 off, double load, verify in DDR
First tile occasionally failed after many reloads	FPGA reprogrammed with a DMA transfer in flight, no PS–PL isolation	Isolation before reprogramming: 25/25 pass vs. failure from the 17th reload

Table 7: Main problems, causes and fixes.

	Content	Status	Evidence
M0	Zynq bring-up: UART, AXI GPIO, own AXI-Lite IP, AXI DMA	Done	DMA loopback 256 B / 64 KB / 1 MB byte-exact
M1	Bicubic baseline, PSNR/SSIM tools, datasets	Done	Baseline in Table 2
M2	FP32 CNN training	Done	+1.04 to +1.53 dB over bicubic, 119/119
M3	INT8 quantization, integer reference model	Done	0.08–0.18 dB drop; golden tiles exported
M4	Convolution engine in RTL	Done	All 4 layers bit-exact; 32 mutations caught
M5	Full network, pixel shuffle, tiling	Done	Seam test bit-exact; timing met
M6	AXI wrapper, DMA, ARM driver, board run	Done	Full 1080p frames byte-exact on board
M7	Benchmarks, reports, reproducible build	In progress	Speed and resources measured
M8	Speed-ups and extensions	Planned	After M7

Table 8: Milestone status.

Remaining for M7: final bitstream with debug counters and repeated board/soak tests; all 119 test images on the board (FPGA quality row); final Vivado reports and a CPU comparison; scripted rebuild of the Vivado project; a demo figure on an image not used in training. **Planned speed-ups (M8):** layer 4 takes 144 of the 203 cycles per pixel, so computing several input channels per cycle there is the largest gain, followed by two pixels in parallel, two INT8 products per DSP, double-buffered DMA and a higher clock; together an estimated 0.1–0.2 s per frame (5–10 fps). Optional extensions: HDMI output, $\times 4$, luma-only processing [7] and ray-traced input [8]. **Review demo (13 Oct 2026):** RTL regression and waveforms in simulation, and a live board run of a golden tile and a full 960×540 frame.

References

[1] C. Dong, C. C. Loy, K. He and X. Tang, "Image super-resolution using deep convolutional networks," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 38, no. 2, pp. 295–307, Feb. 2016.

[2] C. Dong, C. C. Loy and X. Tang, "Accelerating the super-resolution convolutional neural network," in *Proc. ECCV*, 2016, pp. 391–407.

[3] W. Shi *et al.*, "Real-time single image and video super-resolution using an efficient sub-pixel convolutional neural network," in *Proc. IEEE CVPR*, 2016, pp. 1874–1883.

[4] B. Jacob *et al.*, "Quantization and training of neural networks for efficient integer-arithmetic-only inference," in *Proc. IEEE/CVF CVPR*, 2018, pp. 2704–2713.

[5] A. G. Howard *et al.*, "MobileNets: Efficient convolutional neural networks for mobile vision applications," arXiv:1704.04861, 2017.

[6] O. Ahmed, H. Ali, Y. Javed and N. Iqbal, "Super-resolution CNN accelerator on low-cost Zynq-7000 series SoC-based FPGA," in *Proc. URSI 2024*, IEEE, 2024, pp. 257–258.

[7] G. Joseph and E. P. Jayakumar, "Quantized neural network architecture for hardware efficient real-time 4K image super-resolution," in *Proc. 28th Int. Symp. VLSI Design and Test (VDAT)*, 2024, doi: 10.1109/VDAT63601.2024.10705704.

[8] W.-C. Park *et al.*, "An FPGA implementation of Whitted-style ray tracing accelerator," in *Proc. IEEE/EG Symp. Interactive Ray Tracing*, Los Angeles, Aug. 2008, p. 187.

[9] M. Bevilacqua, A. Roumy, C. Guillemot and M.-L. Alberi Morel, "Low-complexity single-image super-resolution based on non-negative neighbor embedding," in *Proc. BMVC*, 2012.

[10] R. Zeyde, M. Elad and M. Protter, "On single image scale-up using sparse-representations," in *Proc. Int. Conf. Curves and Surfaces*, LNCS 6920, 2012, pp. 711–730.

[11] D. Martin, C. Fowlkes, D. Tal and J. Malik, "A database of human segmented natural images and its application to evaluating segmentation algorithms," in *Proc. IEEE ICCV*, 2001, pp. 416–423.

[12] E. Agustsson and R. Timofte, "NTIRE 2017 challenge on single image super-resolution: Dataset and study," in *Proc. IEEE CVPR Workshops*, 2017.

[13] AMD Xilinx, *Zynq-7000 SoC Technical Reference Manual*, UG585.

[14] AMD Xilinx, *AXI DMA LogiCORE IP Product Guide*, PG021.